Google Skills

The solution:

Strategic Model

Understanding model selection

Rule of thumb for model selection:

Start with Gemini 2.5 Pro for prototyping and establishing quality baselines

- Excellent reasoning for complex analysis and quality-critical tasks
- 2M token context window for larger inputs
- Best for initial development and evaluation

Perform gap analysis after switching from Pro to Flash to ensure Flash meets your quality requirements

Optimize with Gemini 2.5 Flash when cost and speed are priorities

- ~2x faster response times
- ~10x cheaper than Pro
- 1M token context window
- Good reasoning for most straightforward tasks
- Ideal for high-volume production workloads

Reference: Gemini model documentation, Vertex AI pricing, Vertex AI model reference

Core concepts

1. Configuring with `GenerateContentConfig`

From the ADK documentation:

> "`generate_content_config` (Optional): Pass an instance of `google.genai.types.GenerateContentConfig` to control parameters like `temperature` (randomness), `max_output_tokens` (response length), `top_p, top_k`, and safety settings."

Python:

```
from google.genai import types

agent = LlmAgent(
    model="gemini-2.5-flash",

    generate_content_config=types.
    GenerateContentConfig(
temperature=0.2,
    # More deterministic output
max_output_tokens=250,
    # Limit response length
top_p=0.8,
    # Nucleus sampling
top_k=10
    # Top-k sampling
    )
)
```

Reference: ADK Docs - Configuring LLM Generation

2. Temperature: Creativity vs. consistency

Temperature controls randomness in model output:

Low temperature (0.0-0.3) - deterministic:

Python:

```
creative_config =
types.GenerateContentConfig(
    temperature=0.9  # More varied,
creative responses
)
# Use for: creative writing,
brainstorming, marketing copy
```

Medium temperature (0.4-0.7) - balanced:

Python:

```
balanced_config =
types.GenerateContentConfig(
    temperature=0.7  # Balance of
creativity and consistency
)
# Use for: customer support, tutoring,
general conversation
```

High temperature (0.8-1.0) - creative:

Python:

```
creative_config =
types.GenerateContentConfig(
    temperature=0.9  # More varied,
creative responses
)
# Use for: creative writing,
brainstorming, marketing copy
```

Reference: ADK Docs - Configuring LLM Generation

3. Safety settings

Configure content filtering thresholds for Gemini models:

Python:

```
from google.genai import types

# Strict safety for public-facing
agents
strict_config =
types.GenerateContentConfig(
    safety_settings=[
        types.SafetySetting(

category=types.HarmCategory.HARM_CATE
GORY_DANGEROUS_CONTENT,

threshold=types.HarmBlockThreshold.BL
OCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(

category=types.HarmCategory.HARM_CATE
GORY_HARASSMENT,

threshold=types.HarmBlockThreshold.BL
OCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(

category=types.HarmCategory.HARM_CATE
GORY_HATE_SPEECH,

threshold=types.HarmBlockThreshold.BL
OCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(

category=types.HarmCategory.HARM_CATE
GORY_SEXUALLY_EXPLICIT,

threshold=types.HarmBlockThreshold.BL
OCK_LOW_AND_ABOVE
        )
    ]
)
```

Safety thresholds:

- `BLOCK_NONE` - No filtering (not recommended for production)
- `BLOCK_ONLY_HIGH` - Block only high-probability harmful content
- `BLOCK_MEDIUM_AND_ABOVE` - Block medium and high probability
- `BLOCK_LOW_AND_ABOVE` - Most strict, blocks even low probability

Note:

For non-Gemini models (e.g., Claude, GPT), refer to the specific model provider's documentation for safety configuration options, as the API and parameters differ from Gemini's safety settings.

Reference: ADK Docs - Safety Settings Example, Gemini Safety Settings

4. Output tokens and sampling

Control response length and diversity:

Python:

```
comprehensive_config = types.GenerateContentConfig(
    temperature=0.7,
    max_output_tokens=2000,
# Allow longer responses
    top_p=0.95,
# Consider top 95% of probability mass
    top_k=40
# Consider top 40 tokens at each step
)


concise_config = types.GenerateContentConfig(
    temperature=0.3,
    max_output_tokens=100,
# Force brief responses
    top_p=0.8,
# More focused sampling
    top_k=10
# Fewer token choices
)
```

Parameters explained:

- `max_output_tokens`: Maximum response length (default varies by model)
- `top_p`: Nucleus sampling—consider tokens comprising top P% of probability
- `top_k`: Only sample from the K most likely next tokens

Reference: ADK Docs - Configuring LLM Generation

Hands-on example

Let's build two agents optimized for different tasks: one for factual data extraction, one for creative brainstorming.

Step 1: Create the project

Shell
```
adk create model_comparison
cd model_comparison
```

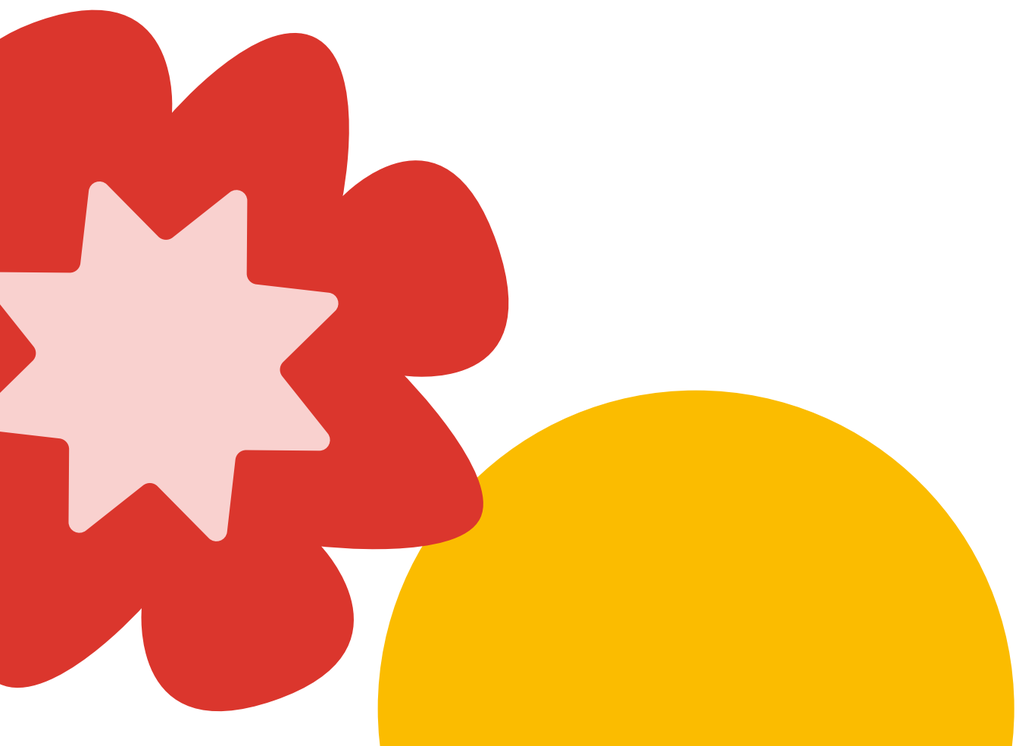
Step 2: Write optimized agents

Replace the contents of agent.py with:

Python
```
"""
Model configuration demonstration showing factual vs creative optimization.
Demonstrates ADK's generate_content_config with different settings.
"""

from google.adk.agents import LlmAgent
from google.genai import types

# Agent 1: Optimized for Factual Data Extraction
# Uses low temperature for consistency, strict safety for accuracy
factual_agent = LlmAgent(
    model="gemini-2.5-flash",  # Flash is sufficient for extraction
    name="data_extractor",
    description="Extracts factual information with high consistency",
    instruction="""You are a precise data extractor.
Extract facts exactly as stated. Do not:
- Add information not present in the input
- Make assumptions or inferences
- Use creative language
```



Step 2: Write the agent cont.

```
Be accurate, concise, and deterministic.""",
    generate_content_config=types.Generate
ContentConfig(
        temperature=0.1,  # Very low
for consistency
        max_output_tokens=500,
        top_p=0.8,
        top_k=10,
        safety_settings=[
            types.SafetySetting(
                category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
                threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
            )
        ]
    )
)

# Agent 2: Optimized for Creative Brainstorming
# Uses high temperature for creativity, Pro model for better ideas
creative_agent = LlmAgent(
    model="gemini-2.5-pro",  # Pro for superior creativity
    name="creative_brainstormer",
    description="Generates creative ideas and explores possibilities",
    instruction="""You are a creative brainstorming partner.

Generate innovative, diverse, and imaginative ideas. Feel free to:
- Think outside the box
- Combine unexpected concepts
- Explore unconventional approaches

Be creative, varied, and thought-provoking.""",
    generate_content_config=types.GenerateContentConfig(
        temperature=0.9,  # High for creativity
        max_output_tokens=2000,  # Allow detailed ideas
        top_p=0.95,
        top_k=40,
        safety_settings=[
            types.SafetySetting(
                category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
                threshold=types.HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE
            )
        ]
    )
)

# For adk web, we'll use the factual agent as root_agent
# Switch to creative_agent to test different behavior
root_agent = factual_agent
```

Step 3: Run and compare

```
Shell
adk web
```

Test with factual agent:

Try this prompt:

```
None
Extract key information: "The iPhone 15 Pro was released in September 2023 and
starts at $999 for 128GB."
```

Expected behavior:

- Consistent, factual response every time
- Extracts exactly what's stated
- No creative interpretation
- Same format on repeated prompts

Test with creative agent:

Change `root_agent = creative_agent` in the code, restart, and try:

```
None
Brainstorm 5 innovative features for a next-generation smartphone.
```

Expected behavior:

- Varied, creative ideas
- Different responses each time
- Imaginative suggestions
- Rich, detailed elaboration

What to notice:

- Temperature impact: Low (0.1) = consistent, High (0.9) = creative
- Model choice: Pro for quality baseline and complex creativity, Flash for cost optimization
- Token limits: Factual agent capped at 500, creative at 2000
- Safety levels: Stricter for factual, balanced for creative

Key takeaways

- Model selection: Start with Pro for prototyping and quality baselines, optimize with Flash for cost and speed
- Temperature: 0.0-0.3 (factual), 0.4-0.7 (balanced), 0.8-1.0 (creative)
- Import from google.genai: `from google.genai import types`
- GenerateContentConfig: Configures temperature, tokens, sampling, and safety
- Safety thresholds: BLOCK_LOW_AND_ABOVE (strict) to BLOCK_ONLY_HIGH (relaxed)
- Optimize for task: Configure parameters based on use case, not one-size-fits-all